

copy 程序

原 copy 程序（增加计时功能）：

```
int main(int argc, char *argv[])
{
    if (argc != 3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1], O_RDONLY); // 读打开已有文件 file1, oldFile 是用来读 file1 的文件描述符
    int newFile = open(argv[2], O_WRONLY | O_CREAT, S_IRUSR | S_IWUSR);
    // 创建一个新文件 file2, newFile 是用来写 file2 的文件描述符

    time_t start = time(NULL);

    char c; // 循环，直至 file1 读完，read 函数返回 0
    while (read(oldFile, &c, 1) == 1) // 从 file1 读一个字节，存入变量 c
        write(newFile, &c, 1); // 将其写入 file2

    time_t end = time(NULL);
    double diff = difftime(end, start);

    printf("Copy Done in %.2f seconds\n", diff); // 人机交互。写屏幕告知程序运行结束。

    close(oldFile); // 文件，访问完及时关闭。这是一个好习惯。
    close(newFile);
}
```

改写的 copy_4096:

```
int main(int argc, char *argv[])
{
    if (argc != 3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1], O_RDONLY); // 读打开已有文件 file1, oldFile 是用来读 file1 的文件描述符
    int newFile = open(argv[2], O_WRONLY | O_CREAT, S_IRUSR | S_IWUSR);
    // 创建一个新文件 file2, newFile 是用来写 file2 的文件描述符

    time_t start = time(NULL);

    char buffer[4096]; // 使用缓冲区一次读取多个字节
    ssize_t bytesRead;
    while ((bytesRead = read(oldFile, buffer, sizeof(buffer))) > 0)
    {
        write(newFile, buffer, bytesRead);
    }

    time_t end = time(NULL);
    double diff = difftime(end, start);

    printf("Copy Done in %.2f seconds\n", diff); // 人机交互。写屏幕告知程序运行结束。

    close(oldFile); // 文件，访问完及时关闭。这是一个好习惯。
    close(newFile);
}
```

为了清楚显示复制时间差异，使用一个较大的文件（莎士比亚全集 word 文档）：

```
D:\desktop\操作系统\思考题1>gcc -o copy copy.c

D:\desktop\操作系统\思考题1>gcc -o copy_4096 copy_4096.c

D:\desktop\操作系统\思考题1>.\copy file1.txt file2.txt
Copy Done in 95.00 seconds

D:\desktop\操作系统\思考题1>.\copy_4096 file1.txt file3.txt
Copy Done in 0.00 seconds
```

file1.txt	2017/6/20 14:33	txtfilelegacy	3,879 KB
file2.txt	2025/9/16 15:13	txtfilelegacy	3,879 KB
file3.txt	2025/9/16 15:14	txtfilelegacy	3,879 KB

可以看到两次复制的时间有明显差异，`copy_4096` 的速度远远快于原程序。这是因为 `read` 和 `write` 都是系统调用函数。当程序执行系统调用时，CPU 会从用户态切换到内核态来执行内核中的代码，完成操作后再切换回用户态。这个切换过程本身需要消耗时间和 CPU 资源。对于原本的程序，每复制一个字节就要进行一轮切换，这个切换过程消耗大量时间。而改写后的程序复制 4096 字节才进行一轮切换，自然会快很多。

方阵自乘程序

1.matrix 程序包括 main 函数、produce 函数和 printf 函数

2. 只读数据: "matrixDes == \n"、"%d\t"、\n"

全局变量: version、matrixOriginal[M][M]、matrixDes [M][M]

局部变量: main 函数内: int i,j

produce 函数内: int row, int column, int i

3.

(1) M 是宏定义。它本身不是一个变量，不占用内存空间。在编译预处理阶段，编译器会自动将代码中所有 M 替换为设定的数值。

(2) 首先是计算量变大了。矩阵乘法时间复杂度是 $O(n^3)$ ，当 M 增加到 4096，计算量大幅增加。还有就是缓存命中率的原因。produce 函数内部循环访问 matrixOriginal 矩阵时是按列访问的，而 C 语言中二维数组在内存里是按行存储的。当 M 变为 4096 时，一行数据占了 4096×4 字节，超过 cache 行容量，导致每一次访问下一行数据时 cache 都未命中，要重新从内存读取，速度就会变慢。

```
#define M 2048

int version = 1;
int matrixOriginal[M][M];
int matrixDes[M][M];
void produce(int row, int column)
{
    int i;
    for (i = 0; i < M; i++)
    {
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
    }
}
```

```

    }
}

int main()
{
    int i, j;

    // 初始化矩阵
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < M; j++)
        {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }
    }

    // 计算矩阵自乘
    time_t start = time(NULL);
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < M; j++)
        {
            produce(i, j);
        }
    }

    time_t end = time(NULL);
    double diff = difftime(end, start);

    // 打印运行时间
    printf("Time taken: %.2f seconds\n", diff);

    return 0;
}

```

实际测试的时候用 4096 太耗费时间了，于是改为 2048，也能看出差距

Time taken: 90.00 seconds

(3) 优化方法：交换循环顺序

在 `matrixDes[i][j] += matrixOriginal[i][k] * matrixOriginal[k][j]` 中，固定 `i` 和 `k`，底层循环变为 `j`（原先是固定 `i` 和 `j`，底层循环是 `k`），这样 `matrixOriginal[k][j]` 是按行访问，`matrixDes[i][j]` 也是按行访问。这样就大大改善了内存访问的局部性，能显著提高缓存命中率，也就能够大幅提高运行速度。

```

#define M 2048

int version = 1;

int matrixOriginal[M][M];
int matrixDes[M][M];

int main()

```

```
{  
  
    int i, j, k;  
  
    // 初始化矩阵  
    for (i = 0; i < M; i++)  
    {  
        for (j = 0; j < M; j++)  
        {  
            matrixOriginal[i][j] = 1;  
            matrixDes[i][j] = 0;  
        }  
    }  
  
    // 计算矩阵自乘  
    time_t start = time(NULL);  
    for (i = 0; i < M; i++)  
    {  
        for (k = 0; k < M; k++)  
        {  
            for (j = 0; j < M; j++)  
            {  
                matrixDes[i][j] += matrixOriginal[i][k] * matrixOriginal[k][j];  
            }  
        }  
    }  
  
    time_t end = time(NULL);  
    double diff = difftime(end, start);  
  
    // 打印运行时间  
    printf("Time taken: %.2f seconds\n", diff);  
    return 0;  
}
```

Time taken: 19.00 seconds